

Claude + Cursor AI Development Workflow

Reusable setup, project onboarding, planning, implementation, review, QA, and parallel work

This guide documents the complete workflow we configured so you can reuse it for future projects without repeating the full setup process.

Reusable reference guide

Claude Code + Cursor + Git worktrees + Markdown handoff

1. The big picture

Use the tools with clear responsibilities:

- **Cursor IDE**: your main development workspace.
- **Cursor Agent / Grok / Composer**: bulk implementation, routine coding, refactors, and fixing review findings.
- **Claude Code**: architecture, planning, project understanding, difficult debugging, review, QA, security, and integration investigation.
- **Markdown files**: the handoff contract between Claude and Cursor.
- **Git branches/worktrees**: isolate parallel tasks so multiple agents can work safely.

The normal flow is:

```
You
|
v
Claude (planning / architecture)
|
v
docs/tasks/TASK-ID.md
|
v
Cursor / Grok (implementation)
|
v
Claude (code review)
|
v
docs/reviews/TASK-ID-REVIEW.md
|
v
Cursor / Grok (fixes)
|
v
Claude QA
|
v
PASS -> commit / PR
```

PART A - ONE-TIME MACHINE SETUP

2. Claude plan

For this workflow, use your **Claude Max** subscription with Claude Code.

Important points:

- Claude Code is authenticated with the Claude subscription account.
- Do not use Anthropic API billing for this workflow unless you intentionally want separate API charges.
- Claude web and Claude Code usage come from the subscription allowance.

3. Install Claude Code on Windows

Open PowerShell and run:

```
irm https://claude.ai/install.ps1 | iex
```

Verify:

```
claude --version
```

If Windows cannot find `claude`, make sure this path is in your user `PATH`:

```
%USERPROFILE%\local\bin
```

Example PowerShell setup:

```
$claudeBin = "$env:USERPROFILE\local\bin"
$userPath = [Environment]::GetEnvironmentVariable("Path", "User")

if (($userPath -split ';') -notcontains $claudeBin) {
    [Environment]::SetEnvironmentVariable(
        "Path",
        ($userPath.TrimEnd(';') + ';' + $claudeBin),
        "User"
    )
}

$env:Path += ";$claudeBin"
```

Close and reopen PowerShell if needed, then verify again:

```
claude --version
```

4. Authenticate Claude Code correctly

Run:

```
claude
```

Choose:

```
Claude account with subscription
```

Do **not** choose Anthropic Console/API billing for the subscription-based workflow.

After login, use:

```
/status
```

Verify that the login method is your Claude Max account.

Also check that no Anthropic API key overrides the subscription:

```
echo $env:ANTHROPIC_API_KEY
```

For this setup, the result should normally be blank.

PART B - MASTER TEMPLATE

5. Master template location

Your reusable master template is stored at:

```
D:\Work\Projects\AI-Project-Template
```

This is the source you reuse for every new or existing project.

Do not rebuild the agents and rules for every project. Copy the master template instead.

6. Master template structure

```
AI-Project-Template/
|
|-- CLAUDE.md
```

```

|-- AGENTS.md
|-- .gitignore
|
|-- .claude/
|   |-- settings.json
|   |-- agents/
|       |-- architect.md
|       |-- explorer.md
|       |-- code-reviewer.md
|       |-- qa-engineer.md
|       |-- security-reviewer.md
|       |-- deep-debugger.md
|       |-- integration-investigator.md
|   |-- hooks/
|       |-- agent-guard.sh
|       |-- test-commands.txt
|
|-- docs/
|   |-- ai/
|       |-- PROJECT_CONTEXT.md
|       |-- ARCHITECTURE.md
|       |-- INTEGRATIONS.md
|       |-- DEVELOPMENT_STANDARDS.md
|       |-- ENVIRONMENTS.md
|       |-- SECURITY_GUIDELINES.md
|   |-- tasks/
|   |-- reviews/
|   |-- investigations/
|
|-- templates/
|   |-- TASK_TEMPLATE.md
|   |-- BUG_TEMPLATE.md
|   |-- INVESTIGATION_TEMPLATE.md
|   |-- REVIEW_TEMPLATE.md
|   |-- PARALLEL_EXECUTION_TEMPLATE.md

```

`settings.local.json` is machine/local-session specific and must not be copied or committed as part of the reusable template.

7. Model routing

Use expensive models only when they add meaningful value.

Role	Model	Use for
Main Claude session	Sonnet 5	Normal coordination and day-to-day reasoning
Architect	Opus 5	Architecture, complex planning, system/database design, critical technical decisions
Deep Debugger	Opus 5	Difficult bugs, race conditions, distributed issues, financial calculation bugs
Code Reviewer	Sonnet 5	Review implementation against approved task
QA Engineer	Sonnet 5	Acceptance criteria, edge cases, regression testing
Security Reviewer	Sonnet 5	Auth, RBAC, payments, secrets, security-sensitive changes
Integration Investigator	Sonnet 5	Shopify, Stripe, GCP, Vercel, webhooks, third-party configuration
Explorer	Haiku 4.5	File discovery, repository search, locating patterns, quick summaries

Key rule

Do **not** keep the main session on Opus all day.

Keep the main session on Sonnet:

```
/model
```

Select Sonnet 5.

The specialist agents keep their own assigned models.

8. Agent safety model

The master template includes technical guards in addition to written instructions.

General behavior:

- Explorer: read/search only.
- Architect: may write planning/docs, not application source code.
- Code reviewer: may write only review documents.
- Security reviewer: review-focused, no production/security mutation.
- QA: may run approved tests; application changes should require approval.
- Deep debugger: diagnostic by default; application edits should require explicit instruction.
- Integration investigator: inspection/read-only by default.

High-risk actions should never be automatically approved, including:

- production deployments
- production database mutations
- payment/refund actions
- changing production webhooks
- DNS changes
- IAM/security permission changes
- deleting cloud resources
- secret rotation
- publishing ecommerce themes/configuration
- sending real customer communications
- destructive Git operations

PART C - DAILY IDE WORKFLOW

9. Use Cursor as the main workspace

You do not need a standalone PowerShell window for normal development.

Open the project in **Cursor** and use Cursor's integrated terminal.

Typical layout:

```
Cursor IDE
|
|-- Editor / code explorer
|-- Cursor Agent / Grok
|
|-- Integrated terminals
|   |-- Terminal 1: claude
|   |-- Terminal 2: npm run dev
|   |-- Terminal 3: backend server
|   |-- Terminal 4: git / tests
```

For actual project work, launch Claude from the correct project root:

```
cd "D:\Work\Projects\YourProject"
claude
```

Where Claude starts determines what repository/workspace context it can read.

PART D - APPLY THE TEMPLATE TO A PROJECT

10. Existing single-repository project

Example:

```
MyProject/  
|-- .git/  
|-- src/  
|-- package.json  
`-- ...
```

Copy the master AI files into the project root, but never copy the template's `.git` folder or `settings.local.json`.

Example PowerShell:

```
$src = "D:\Work\Projects\AI-Project-Template"  
$dst = "D:\Work\Projects\MyProject"  
  
Copy-Item "$src\CLAUDE.md" "$dst\CLAUDE.md"  
Copy-Item "$src\AGENTS.md" "$dst\AGENTS.md"  
robocopy "$src\.claude" "$dst\.claude" /E /XF settings.local.json  
robocopy "$src\docs" "$dst\docs" /E  
robocopy "$src\templates" "$dst\templates" /E
```

Before copying into an existing project, first check for collisions so you do not overwrite existing project-specific files.

11. New project

For a new project, you can copy the master template first and then build the application inside the same repository.

Recommended sequence:

1. Create project folder
2. Copy master template
3. Initialize Git / application framework
4. Run Claude from project root
5. Populate project context
6. Begin task workflow

12. Multi-repository workspace

Example:

```
ProductWorkspace/  
|  
|-- CLAUDE.md  
|-- AGENTS.md  
|-- .claude/  
|-- docs/  
|-- templates/  
|  
|-- frontend/    <- independent Git repo  
|-- backend/     <- independent Git repo  
`-- admin/       <- independent Git repo
```

Run Claude from the **parent workspace** so it can understand the whole product:

```
cd "D:\Work\Projects\ProductWorkspace"  
claude
```

Do **not** initialize Git at the parent root if the child applications already have independent Git repositories unless you intentionally want that structure.

The root becomes the AI coordination layer; each child repository keeps its own Git history.

PART E - ONBOARD AN EXISTING PROJECT

13. Read existing documentation first

If an existing project already has README files, `docs/`, plans, TODOs, architecture notes, status files, or deployment docs, Claude should read those **before** broadly analyzing the codebase.

Recommended order:

```
Existing docs
|
v
Understand architecture / decisions / completed work / pending work
|
v
Selective code verification
|
v
Create consolidated AI context
```

Do not make Claude rediscover everything from source code if the project documentation already explains it.

14. Populate project-specific AI context

For an existing project, Claude should populate:

```
docs/ai/PROJECT_CONTEXT.md
docs/ai/ARCHITECTURE.md
docs/ai/INTEGRATIONS.md
docs/ai/DEVELOPMENT_STANDARDS.md
docs/ai/ENVIRONMENTS.md
docs/ai/SECURITY_GUIDELINES.md
```

For active projects, also create/use:

```
docs/ai/PROJECT_STATUS.md
```

Suggested status structure:

```
# Project Status

## Frontend
### Completed
### In Progress
### Pending
### Known Issues

## Backend
### Completed
### In Progress
### Pending
### Known Issues

## Admin
### Completed
### In Progress
### Pending
### Known Issues

## Cross-Application Work
### Completed
### In Progress
```

Pending

Do not guess project status only from the existence of source code. Prefer explicit project documentation and verified evidence.

PART F - NORMAL TASK WORKFLOW

15. Step 1 - Give Claude the requirement

Example:

```
We need to add website scanning where a user enters a URL and we analyze SEO metadata, Open Graph tags, sitemap, robots.txt, and AI-search readiness.
```

Do not immediately ask Cursor/Grok to code a complex feature.

16. Step 2 - Claude creates the implementation plan

For medium/large work, ask Claude to use the architect agent.

Example:

```
Use the architect agent.

Analyze the existing project and create an implementation plan for this feature.
Do not implement application code.

Create:
docs/tasks/AIW-101-WEBSITE-SCANNER.md
```

The task file should normally include:

- objective
- business requirement
- current behavior
- expected behavior
- scope
- out of scope
- architecture decision
- database changes
- backend changes
- frontend changes
- API changes
- integrations
- security considerations
- edge cases
- implementation sequence
- tests
- acceptance criteria
- likely files affected
- dependencies

This Markdown file becomes the **approved contract** for implementation.

17. Step 3 - Cursor/Grok implements the plan

In Cursor Agent, use a prompt like:

```
Read :
@docs/tasks/AIW-101-WEBSITE-SCANNER.md
```

Also read:
@AGENTS.md

Implement the approved specification.
Follow the existing project architecture.
Do not redesign the approved solution without explaining why.
Do not make unrelated changes.
Run relevant tests when complete.

Cursor/Grok is now the developer, not the architect.

18. Step 4 - Claude reviews the implementation

After Cursor completes the task:

Implementation for docs/tasks/AIW-101-WEBSITE-SCANNER.md is complete.

Use the code-reviewer agent.
Review the implementation against the approved specification.
Inspect the relevant git diff.
Do not modify application code.

Create:
docs/reviews/AIW-101-REVIEW.md

Review classifications:

BLOCKER
MAJOR
MINOR
OPTIONAL

19. Step 5 - Cursor fixes review findings

Example:

Read:
@docs/reviews/AIW-101-REVIEW.md

Fix all BLOCKER and MAJOR findings.
Do not make unrelated changes.
Run relevant tests.

20. Step 6 - Claude QA

Example:

Use the QA engineer.

Validate the current implementation against:
docs/tasks/AIW-101-WEBSITE-SCANNER.md

Check acceptance criteria, happy paths, failure paths, boundary cases, regressions, and required tests.

Expected result:

PASS

or:

PARTIAL
- issue 1
- issue 2

or:

```
FAIL
- blocker details
```

If needed, send the failed QA findings back to Cursor for fixes.

21. Step 7 - Commit / PR

Only after review and QA are clean:

```
Task plan -> implementation -> review -> fixes -> QA -> commit / PR
```

PART G - WHEN TO USE EACH AGENT

22. Architect - Opus

Use when the task requires:

- system architecture
- complex feature planning
- database redesign
- cross-service changes
- billing/payment architecture
- authentication/authorization architecture
- large refactors
- difficult technical decisions

Avoid using it for tiny CRUD tasks or obvious code changes.

23. Explorer - Haiku

Use for:

- finding files
- locating existing patterns
- tracing code paths
- identifying dependencies
- quick codebase summaries

Example:

```
Use the explorer agent to find where invoice generation is implemented and list the relevant files and call flow.
```

24. Code Reviewer - Sonnet

Use after implementation.

Checks:

- correctness
- regressions
- architecture compliance
- maintainability
- performance
- validation
- error handling
- missing tests

25. QA Engineer - Sonnet

Use to validate:

- acceptance criteria
- happy path
- failure path
- boundary cases
- regression risks
- missing tests

26. Security Reviewer - Sonnet

Use only when relevant, for example:

- authentication
- authorization/RBAC
- payments
- billing
- webhooks
- uploads
- tokens
- secrets
- PII
- admin permissions
- infrastructure security

Do not run the security reviewer automatically for every simple UI task.

27. Deep Debugger - Opus

Use when normal debugging has failed, especially for:

- race conditions
- distributed systems behavior
- difficult production-only bugs
- complex state problems
- financial calculation bugs
- hard integration failures

It should investigate root cause before proposing changes.

28. Integration Investigator - Sonnet

Use for issues involving external systems such as:

- Shopify
- Stripe
- GCP
- AWS
- Vercel
- Cloudflare
- HubSpot
- Voiceflow
- PostHog
- webhooks
- SaaS dashboards

Its job is to determine whether the problem is:

```
application code
configuration
third-party behavior
environment
permissions
```

infrastructure

Investigation should be read-only by default.

PART H - SIMPLE VS COMPLEX TASKS

29. Tiny task

Example:

Add a phone field to a profile form.

Workflow:

Cursor/Grok -> implement -> tests -> optional quick review

Do not create five agents for a two-line change.

30. Medium task

Example:

Add seller offers.

Workflow:

Claude Architect -> TASK.md -> Cursor implementation -> Claude review -> fixes

31. Critical task

Example:

Implement payment payouts or authentication changes.

Workflow:

```
Claude Architect / Opus
|
Security review
|
TASK.md
|
Cursor implementation
|
Claude code review
|
Claude QA
|
Cursor fixes
|
Final review
```

PART I - PARALLEL TASKS

32. Can five tasks run at the same time?

Yes, but only when they are isolated and dependencies are understood.

Do not allow five agents to edit the same working directory.

Preferred pattern:

```
Task 1 -> Branch/worktree 1
Task 2 -> Branch/worktree 2
Task 3 -> Branch/worktree 3
Task 4 -> Branch/worktree 4
Task 5 -> Branch/worktree 5
```

Before parallelizing, ask Claude Architect to create a parallel execution plan.

Example:

```
Analyze these five tasks and determine:
- dependencies
- overlapping files
- shared APIs
- database migrations
- shared external systems
- safe parallel groups
- recommended merge order

Create:
docs/tasks/PARALLEL_EXECUTION_PLAN.md
```

33. Git worktrees

Use worktrees when multiple tasks need to run simultaneously against the same repository.

Concept:

```
main repository
|
|-- worktree-task-101
|-- worktree-task-102
|-- worktree-task-103
|-- worktree-task-104
`-- worktree-task-105
```

Each agent/session works in a separate worktree/branch.

Do not blindly parallelize dependent tasks.

34. Things worktrees do NOT isolate

Git worktrees isolate files and branches, but not external/shared systems.

Watch out for:

- shared development database
- shared Stripe test account
- shared Shopify dev store
- shared cloud development environment
- shared ports
- shared third-party webhooks

Example port allocation:

```
Task 1 -> frontend 3001 / backend 4001
Task 2 -> frontend 3002 / backend 4002
Task 3 -> frontend 3003 / backend 4003
```

For database-sensitive parallel work, use isolated databases/schemas where practical or explicitly block destructive shared-state changes.

PART J - MARKDOWN FILES AS THE HANDOFF CONTRACT

35. Why task/review Markdown files matter

Do not rely on copy/pasting conversational instructions between Claude and Cursor.

Use permanent files:

```
docs/tasks/
|-- AIW-101-WEBSITE-SCANNER.md
|-- AIW-102-SEO-AUDIT.md
`-- AIW-103-SUBSCRIPTION.md

docs/reviews/
|-- AIW-101-REVIEW.md
`-- AIW-102-REVIEW.md

docs/investigations/
`-- AIW-104-INTEGRATION-ISSUE.md
```

Benefits:

- Claude and Cursor read the same requirements.
- Plans survive chat/session changes.
- Review findings remain traceable.
- New developers/agents can understand why code changed.
- Parallel tasks have clear contracts.

PART K - PROJECT DOCUMENTATION RULES

36. Existing project docs vs root AI docs

For an established project, do not replace existing detailed documentation unnecessarily.

Use:

```
application/docs/  -> detailed app-specific documentation
root/docs/ai/      -> concise cross-project AI context
```

The root AI context should reference deeper documentation rather than copy every detail.

37. Keep project context current

When a major architecture decision changes, update the relevant project context.

Examples:

```
docs/ai/ARCHITECTURE.md
docs/ai/INTEGRATIONS.md
docs/ai/PROJECT_STATUS.md
```

Do not update these files for every tiny code edit. Update them when the project's actual architecture/status meaningfully changes.

PART L - NEW PROJECT CHECKLIST

38. Quick checklist for a new project

```
[ ] Open project in Cursor
[ ] Determine single repo or multi-repo workspace
```

```
[ ] Check whether CLAUDE.md/.claude/docs/templates already exist
[ ] Copy master template safely
[ ] Do not copy template .git
[ ] Do not copy settings.local.json
[ ] Launch Claude from correct project/workspace root
[ ] Keep main model on Sonnet
[ ] Read existing docs first (for existing projects)
[ ] Populate docs/ai project context
[ ] Confirm Git structure is untouched
[ ] Start feature work using TASK.md handoff
```

PART M - EXISTING PROJECT CHECKLIST

39. Quick checklist for an existing project

Before copying:

```
@(".claude", "CLAUDE.md", "AGENTS.md", "docs", "templates") | ForEach-Object {
    "$_ = $(Test-Path $_)"
}
```

For a workspace, identify nested Git repos:

```
Get-ChildItem -Directory | ForEach-Object {
    if (Test-Path (Join-Path $_.FullName ".git")) {
        "$($_.Name) = Git repository"
    } else {
        "$($_.Name) = NOT a Git repository"
    }
}
```

Then copy safely and let Claude read existing documentation before broad code analysis.

PART N - DAILY QUICK REFERENCE

40. Start Claude in a project

```
cd "D:\Work\Projects\ProjectName"
claude
```

Check status:

```
/status
```

Change main model:

```
/model
```

Normal default: **Sonnet**.

41. Planning prompt

```
Use the architect agent.
Analyze the existing implementation and create a detailed implementation plan.
Do not modify application code.
Create docs/tasks/TASK-ID-NAME.md.
```

42. Cursor implementation prompt

```
Read @docs/tasks/TASK-ID-NAME.md and @AGENTS.md.  
Implement the approved specification.  
Follow existing architecture.  
Do not make unrelated changes.  
Run relevant tests.
```

43. Review prompt

```
Use the code-reviewer agent.  
Review the implementation against docs/tasks/TASK-ID-NAME.md.  
Inspect the relevant git diff.  
Do not modify application code.  
Create docs/reviews/TASK-ID-REVIEW.md.
```

44. QA prompt

```
Use the QA engineer.  
Validate the implementation against docs/tasks/TASK-ID-NAME.md.  
Check acceptance criteria, failure paths, edge cases, regressions, and tests.
```

45. Integration investigation prompt

```
Use the integration-investigator agent.  
Determine whether this problem is caused by application code, configuration, third-party behavior,  
environment, permissions, or infrastructure.  
Inspect only; do not make production changes.  
Create docs/investigations/ISSUE-ID.md.
```

46. Deep debugging prompt

```
Use the deep-debugger agent.  
Normal debugging has failed.  
Investigate root cause before proposing changes.  
Do not make unrelated changes.
```

PART O - RULES TO REMEMBER

47. The core rules

1. **Claude plans; Cursor implements.** Claude can code when needed, but this separation keeps usage and responsibilities clean.
2. **Keep the main Claude session on Sonnet.** Use Opus through specialist agents for high-value reasoning.
3. **Use Markdown plans as contracts.** Do not rely on chat history alone.
4. **Read existing project docs first.** Do not waste tokens rediscovering documented decisions.
5. **Never let multiple agents edit the same working tree simultaneously.** Use branches/worktrees.
6. **Do not automatically modify production systems.** Read/inspect first; require explicit approval for consequential actions.
7. **Do not store secrets in AI documentation.** Never put API keys, passwords, tokens, credentials, or sensitive customer data in these files.
8. **Do not create an extra Git repo at a multi-repo parent unless intentionally designed.**
9. **Use simple workflows for simple tasks.** Do not spawn expensive agents unnecessarily.
10. **Review and QA before final commit/PR for meaningful changes.**

PART P - YOUR CURRENT REUSABLE PATHS

48. Master template

```
D:\Work\Projects\AI-Project-Template
```

49. Example multi-repository workspace

```
D:\Work\Projects\AiWeblyzer
```

Current AiWeblyzer structure:

```
AiWeblyzer/  
|-- .claude/  
|-- CLAUDE.md  
|-- AGENTS.md  
|-- docs/  
|-- templates/  
|  
|-- aiweblyzer-admin-app/      <- independent Git repository  
|-- aiweblyzer-backend-service/ <- independent Git repository  
|-- aiweblyzer-frontend-app/   <- independent Git repository  
`-- aiweblyzer.code-workspace
```

Claude should run from the AiWeblyzer parent root when planning product-wide changes so it can understand all three applications together.

FINAL SUMMARY

For future projects, you do **not** repeat the full installation and agent setup.

Your reusable process is:

1. Open project/workspace in Cursor
2. Determine Git/repository structure
3. Check for file collisions
4. Copy AI-Project-Template safely
5. Run Claude from the correct root
6. Read existing documentation first
7. Populate project context
8. Claude creates TASK.md
9. Cursor/Grok implements
10. Claude reviews
11. Cursor fixes
12. Claude QA
13. Commit / PR

That is the complete reusable Claude + Cursor development workflow.